

BSc Computer Science  
COC251  
F330257

**TinyML**  
**Gesture-Based Text Input**  
**on the Edge**

by

Kishan A. Dave

Supervisor: Dr. P. Tso

Department of Computer Science  
Loughborough University

Submitted: April 21st, 2026

# Abstract

This project investigates model compression techniques for deploying a gesture-based text input system on the Arduino Nano 33 BLE Sense, a microcontroller with 256KB of SRAM and 1MB of Flash. Air-writing offers a keyboard-free input method for wearable and embedded devices. A preliminary investigation confirmed that small, fast gestures (3cm, 0.4s) achieve statistically equivalent classification accuracy to larger, slower gestures, validating their use for practical text input. A dataset of 2,600 samples covering 26 uppercase characters was collected from 6-axis IMU data and used to train a baseline 1D CNN achieving 98.27% accuracy. Structured pruning, post-training int8 quantisation, and knowledge distillation were applied individually and in combination, producing 19 compressed model variants evaluated across accuracy, model size, inference latency, and tensor arena memory usage. Of the techniques evaluated, knowledge distillation into a student architecture with sufficient capacity produced the most favourable performance results, with the distilled student achieving 95.96% accuracy at 67ms latency in 66.9KB, a 71% size reduction over the baseline. Combining distillation with int8 quantisation further reduced the tensor arena footprint to 5.9KB with 91.73% accuracy and 24ms latency, suitable for extremely memory constrained devices. The results demonstrate the feasibility for air-writing recognition on MCUs and the importance of model optimisation choice in TinyML contexts.

# Acknowledgements

I would like to thank my supervisor, Dr. Posco Tso, for his advice and guidance, and for providing me direction throughout this project. I would also like to thank my family for their unwavering support and encouragement.

# Contents

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>Abstract</b>                                     | <b>i</b>  |
| <b>Acknowledgements</b>                             | <b>ii</b> |
| <b>1 Introduction</b>                               | <b>1</b>  |
| 1.1 Aims & Objectives . . . . .                     | 1         |
| 1.2 Limitations . . . . .                           | 1         |
| <b>2 Literature Review</b>                          | <b>2</b>  |
| 2.1 Introduction . . . . .                          | 2         |
| 2.2 Gesture Recognition and Air-Writing . . . . .   | 2         |
| 2.3 TinyML Landscape . . . . .                      | 3         |
| 2.4 Optimisation Techniques . . . . .               | 4         |
| 2.5 Summary . . . . .                               | 5         |
| <b>3 Preliminary Investigation</b>                  | <b>6</b>  |
| 3.1 Purpose . . . . .                               | 6         |
| 3.2 Methodology . . . . .                           | 6         |
| 3.2.1 Data Collection Protocol . . . . .            | 6         |
| 3.2.2 Data Collection Software . . . . .            | 7         |
| 3.2.3 Hypothesis Testing . . . . .                  | 8         |
| 3.3 Results and Analysis . . . . .                  | 9         |
| 3.4 Outcome and Revised Project Direction . . . . . | 10        |
| <b>4 Data Acquisition</b>                           | <b>11</b> |
| 4.1 Dataset Requirements . . . . .                  | 11        |
| 4.1.1 Character Set and Sample Number . . . . .     | 11        |
| 4.1.2 Data Labelling and Organisation . . . . .     | 11        |
| 4.1.3 Data Recording Protocol . . . . .             | 12        |
| 4.2 Data Creation Scripts . . . . .                 | 12        |
| 4.3 Dataset Overview . . . . .                      | 14        |
| <b>5 Model Training and Compression</b>             | <b>16</b> |
| 5.1 Baseline Model . . . . .                        | 16        |
| 5.2 Model Compression Methodology . . . . .         | 17        |

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| 5.2.1    | Structured Pruning . . . . .                             | 17        |
| 5.2.2    | Quantisation . . . . .                                   | 18        |
| 5.2.3    | Knowledge Distillation . . . . .                         | 19        |
| <b>6</b> | <b>Model Testing and Evaluation</b>                      | <b>21</b> |
| 6.1      | Performance Results . . . . .                            | 21        |
| 6.1.1    | Accuracy vs Size Across Compression Techniques . . . . . | 21        |
| 6.1.2    | Model Selection . . . . .                                | 24        |
| 6.1.3    | Testing methodology . . . . .                            | 25        |
| 6.1.4    | On-device Performance of Selected Models . . . . .       | 26        |
| 6.2      | Evaluation of Model Trade-offs . . . . .                 | 30        |
| <b>7</b> | <b>Conclusion</b>                                        | <b>33</b> |
| 7.1      | Summary and Key Findings . . . . .                       | 33        |
| 7.2      | Reflection on Project Aims . . . . .                     | 33        |
| 7.3      | Recommendations for Future Work . . . . .                | 34        |

# 1 Introduction

Emerging technology platforms such as augmented and virtual reality headsets and smart-bands present a common challenge: they require text input but lack keyboards and touchscreens modern platforms feature. A similar problem is faced by those with limited mobility, traditional input devices may be impractical or unusable in some environments. Air-writing, in which a user draws characters in free space as though writing on an imaginary blackboard, offers one potential solution. By capturing hand motion through inertial sensors and classifying character gestures using a machine learning model deployed directly on a microcontroller, text input can be performed without any physical surface, external device, or cloud connectivity.

## 1.1 Aims & Objectives

The aim of this project is to develop and evaluate a TinyML air-writing recognition convolutional neural network deployed on the Arduino Nano 33 BLE Sense.

The specific objectives are:

- (1) investigate whether small, fast gestures can achieve classification accuracy comparable to the larger, slower gestures used in prior work, validating their use for practical text input
- (2) create a dataset of 26 air-written characters using small, fast gestures captured from 6-axis IMU data
- (3) train a baseline 1D CNN model and apply post-training quantisation, structured pruning, and knowledge distillation individually and in combination
- (4) evaluate the compressed models on-device across accuracy, model size, RAM usage, and inference latency to determine which compression strategies offer the most effective tradeoffs for microcontroller deployment for the use case.

## 1.2 Limitations

This study is scoped to a single hardware target, the Arduino Nano 33 BLE Sense, and a single model architecture. All gesture data was recorded by one user, meaning results reflect user-dependent classification and may not generalise across users. The system recognises isolated characters only and does not address continuous handwriting or word recognition from a continuous data stream.

## 2 Literature Review

### 2.1 Introduction

This review draws on literature from three areas central to the thesis: air-writing recognition, the TinyML landscape, and model compression techniques for edge deployment on constrained devices. It aims to provide context for the work done in th thesis and justifies decisions made such as the use of Keras and TensorFlow Lite for model training and deployment, as well as the use of 1D convolutional neural networks for classification.

### 2.2 Gesture Recognition and Air-Writing

Through gesture recognition, human body motion is translated into distinct outputs that can be used to trigger digital actions. Offering an alternative input modality in situations where traditional devices such as touchscreens or remote controls are impractical, with applications in augmented and virtual reality control, sterile environments and accessibility for those with limited mobility.

Air-writing recognition is a subproblem of gesture recognition, in which free space hand movement in the form of drawing characters, as though writing on a blackboard, is translated into text, providing a general text-input method for the growing class of devices such as smart-bands, wearables, and AR/VR headsets that lack keyboards or screens (Yanay & Shmueli, 2020). Prior work has established air-writing as a practical text-input method. Chen et al. (2016) show air-writing achieves relative text-input speed to other motion based methods such as on-screen virtual keyboards operated by pointing devices and Amma et al. (2014) further demonstrate continuous, sentence recognition with large vocabularies is achievable using wearable inertial sensors.

Air-writing recognition has been achieved through different approaches broadly divided into vision-based, using cameras or depth sensors, and sensor-based, which attach sensors directly to the user (Cheok et al., 2019). Of sensor based approaches, using inertial measurement units (IMUs), combining accelerometers and gyroscopes, has been a common choice due to their low cost and power consumption, whilst avoiding constraints of vision-based methods like lighting and line-of-sight.

Recent work has focused on deep-learning, with CNN architectures learning the hierarchical feature representations directly from raw sensor data. Al Abir et al. (2021) demonstrated that a 2D CNN applied to interpolated IMU time-series data could achieve up to 99.15% accuracy on seven public datasets, and Yanay and Shmueli (2020) report 83.2% user-independent classification accuracy for a 26-character set using a 1D CNN on smart-band IMU data, reaching 95.6% accuracy with an

auto-correction phase.

Following this, this thesis adopts a 1D convolutional neural network applied directly to 6-axis IMU data for air-writing classification.

## 2.3 TinyML Landscape

Tiny Machine Learning (TinyML) refers to the practice of deploying machine learning models on highly resource-constrained edge devices such as microcontrollers. By enabling on-device inference, TinyML can offer significant advantages over cloud alternatives such as reduced latency, improved privacy, lower energy consumption, and operation without network connectivity. These benefits have enabled applications including predictive maintenance of machinery, wearable health sensing, and smart agriculture.

The resource budgets of TinyML applications are bounded by RAM, flash memory and energy consumption. For always-on applications such as wearable health monitoring devices powered by small batteries, energy consumption for each inference must be minimised to support operation over many days without recharging. Applications performing keyword spotting must complete inference within milliseconds to avoid perceptible delay. Model size, inference latency and energy consumption are all closely tied to model complexity, which is typically correlated with accuracy; smaller and faster models generally achieve lower accuracy than larger, more complex models. This is the core trade-off of TinyML: achieving acceptable accuracy within the resource constraints of the edge device and within the tolerance for inference latency required by the use case. The choice of appropriate model compression techniques, discussed in the following section, is therefore essential to developing a functional TinyML application.

Microcontrollers (MCUs) widely used in TinyML span a wide performance range. Extremely constrained devices, such as AVR RISC microcontrollers, offer as little as 2KB of SRAM, whereas a typical Cortex-M class microcontroller provides approximately 128KB of SRAM and 1MB of flash (Saha et al., 2022). The Arduino Nano 33 BLE Sense fits on the higher end of the spectrum, containing an Arm Cortex-M4F processor core with 1MB Flash and 256KB RAM and features a large set of onboard sensors: accelerometer, gyroscope, magnetometer, microphone, temperature, etc (Arduino, 2026). This makes it suitable for a diverse range of TinyML applications, and it has consequently been adopted as a target platform in numerous studies i.e. Viswanatha et al. (2022). Given these capabilities and its established use in prior TinyML research, the Arduino Nano 33 BLE Sense is a suitable platform for this thesis.

Frameworks utilised in TinyML facilitate model architecture design, training, compression and deployment. TensorFlow Lite for Microcontrollers (TFLM) is a lightweight inference engine designed

for 32-bit microcontrollers with limited memory. TFLM bypasses operating system-level dynamic memory allocation and instead uses a statically allocated tensor arena, reusing intermediate buffers during inference execution (David et al., 2021). It supports the execution of float32 models, as well as quantised int8 models while keeping compiled firmware minimal. Edge Impulse provides an alternative cloud-based platform covering data collection, training and deployment built upon TFLM. While its compiler has been shown to reduce SRAM usage by 25-55% and flash by 35% compared to standard TFLM deployment (Saha et al., 2022), it limits control over model architecture and compression. Since this project evaluates compression techniques over a range of parameters, the TensorFlow ecosystem is used for the end-to-end pipeline, with keras for training, TFLite for model conversion and TFLM for deployment, to provide the necessary flexibility Edge Impulse lacks..

## 2.4 Optimisation Techniques

Applying quantisation to a model reduces the numerical precision of its weights and activations, for example from float32 to int8, reducing model size (Jacob et al., 2018). This is effective because typically the lower-order decimal places for each weight value are too small to have a significant impact on the output of a neuron. When this is applied to all weights the aggregate loss in precision may result in a significant drop in accuracy, particularly if the weight size is aggressively reduced i.e. from float32 to int4. Quantisation can be applied after traditional model training; full precision weights are mapped to lower-precision values by a scaling factor. Activation values are computed dynamically during inference and their ranges depend on the input data, so calibration samples must be run through the model to determine scaling factors for each layer (Jacob et al., 2018). Alternatively quantisation aware training may be used, in which during model training the effects of quantisation are simulated during the forward pass resulting in weight values more optimised to quantisation, after which quantisation is applied to the model. (Lamaakal et al., 2025)

Pruning is removing parameters from a trained neural network to reduce its size. Unstructured pruning removes individual weights that contribute to the final model output the least under some threshold; at a 30% sparsity threshold, unstructured pruning would remove 30% of the weights of a model that are most redundant. This produces a sparse model, with much fewer non-zero parameters, however for storage and inference benefits the model must be saved in a sparse format and deployed on hardware with sparse inference support. Without a sparse storage format, pruned weights are simply set to zero and remain in the weight matrix, providing no reduction in model size. Structured pruning instead removes entire filters or channels ranked by an importance criterion such as the L1-norm of filter weights (Li et al., 2017), instead of individual weights. Therefore structured pruning produces a fully dense model with size and computation advantages without requiring sparse matrix inference support in the hardware and software platforms used, making it a practical choice for microcontroller deployment. Models are consequently fine tuned to recover lost accuracy.

Knowledge distillation is the process of training a smaller, student model on the behaviour of a larger, teacher model (Hinton, Vinyals and Dean, 2015). The teacher’s output probabilities are softened using a temperature parameter, resulting in an output that reveals which classes the teacher considers similar in addition to which class it would classify the input as. The student is then trained with a combined loss of cross entropy against true labels, and the difference between the student’s softened outputs and the teacher’s. This can allow the student to achieve higher accuracy than if trained independently on the same data, while reducing model size to fit on constrained hardware.

## **2.5 Summary**

With this review, we have established the landscapes of air writing recognition and TinyML, and explained how 3 core model optimisation techniques achieve lower model size. We have also found the application of CNNs trained on IMU data a viable and modern approach to air writing recognition, including when deployed on constrained edge devices.

Two areas of investigation for this thesis were identified from this review. First, prior air-writing systems have typically used large gestures performed over several seconds, and it is unclear whether small, fast gestures suitable for practical text input can achieve comparable classification accuracy. This question is addressed by the preliminary investigation in Chapter 3. Second, quantisation, pruning, and knowledge distillation have each been applied to deploy models on microcontrollers, the relative effectiveness of these techniques for IMU-based character recognition on constrained hardware forms the focus of this thesis and is addressed in Chapters 6 and 7.

## 3 Preliminary Investigation

### 3.1 Purpose

The convenience of the application is determined in part by the size and speed expected of the gesture. For users with limited mobility, or use cases in which lots of text must be input, slow and large gestures may prove impractical. To determine if gestures smaller and faster than those often studied could be picked up by a machine learning model, a feasibility test was necessary.

The primary concern was whether the Inertial Measurement Unit (IMU) data from the Arduino Nano 33 BLE Sense, would maintain a sufficient signal-to-noise ratio for a model to classify characters from the data when training and testing subtle, sub-5cm gestures. This is important because low-cost MEMS sensors are known to suffer from 'accuracy degradation with time due to the combined effects of errors like noises, biases, drifts and scale factor instabilities.' (Aggarwal et al., 2008, p.324).

This investigation aimed to empirically test the following null hypothesis.

$H_0$ : There is a statistically significant difference between the classification accuracy of a 1D CNN trained on IMU time series data of air-written character gestures recorded at a 100Hz sampling rate, when comparing large, slow gestures (20cm, 2 seconds) against small, fast gestures (3cm, 0.4 seconds).

With an alternative hypothesis of

$H_1$ : There is no statistically significant difference between the classification accuracy of a 1D CNN trained on IMU time series data of air-written character gestures recorded at a 100Hz sampling rate, when comparing large, slow gestures (20cm, 2 seconds) against small, fast gestures (3cm, 0.4 seconds).

### 3.2 Methodology

A controlled experiment was designed to isolate and test the variables of gesture size and execution speed. The methodology consisted of two phases: data collection and hypothesis testing.

#### 3.2.1 Data Collection Protocol

Two datasets were created, each containing five distinct characters ('3', 'A', 'C', 'e', 'r') chosen for their gestural distinctness. Fifty repetitions of each character were recorded by a single user (the

author) to ensure that other variables that may affect the results are controlled, such as orientation and shape of gesture.

**Large-Slow (Baseline):** Gestures approximately 20cm in height, executed deliberately over a 2-second period (200 samples at 100 Hz).

**Small-Fast:** Gestures of 3cm height executed at a natural speed of 0.4 seconds (capturing 40 samples at 100 Hz).

Data was captured using the onboard LSM9DS1 IMU of the Arduino Nano 33 BLE Sense, streaming 6-axis data (accelerometer and gyroscope) at 100 Hz to a laptop via a serial connection. The raw data (linear acceleration in  $\text{m/s}^2$  and angular velocity in degrees/s) was time-stamped and saved directly to CSV files.

### 3.2.2 Data Collection Software

A pair of synchronised scripts were developed: one for the microcontroller and one for the host computer. The Arduino script (Listing 3.1) was designed for low-latency sampling. Upon receiving a trigger command ('R'), it collected a fixed number of samples (200 for slow gestures, 40 for fast gestures) at the specified 100 Hz rate before stopping.

```
1 // ... setup code ...
2 void loop() {
3     if (Serial.available() > 0) {
4         char command = Serial.read();
5         if (command == 'R') {
6             int samplesRead = 0;
7             unsigned long lastSampleTime = micros();
8             unsigned long gestureStartTime = micros();
9
10            while (samplesRead < TOTAL_SAMPLES) { // TOTAL_SAMPLES = 40 or 200
11                unsigned long currentTime = micros();
12                if (currentTime - lastSampleTime >= SAMPLE_DELAY) {
13                    if (IMU.accelerationAvailable() && IMU.gyroscopeAvailable()) {
14                        float ax, ay, az, gx, gy, gz;
15                        IMU.readAcceleration(ax, ay, az);
16                        IMU.readGyroscope(gx, gy, gz);
17                        // Convert acceleration to m/s^2 and print all data
18                        Serial.print(ax * 9.80665, 4); Serial.print(",");
19                        // ... (remaining axes) ...
20                        Serial.println(currentTime - gestureStartTime);
21                        samplesRead++;
22                        lastSampleTime = currentTime;
23                    }
24                }
25            }
26        }
27    }
28 }
```

```

25     }
26     Serial.println("END_OF_GESTURE");
27 }
28 }
29 }

```

Listing 3.1: Core Arduino data collection loop for fixed-length recording.

Data collection was managed by a custom Python script running on the host computer. The script initiated each recording session by transmitting a trigger byte ('R') to the Arduino via the serial port. It then buffered the incoming data stream, parsed the 6-axis sensor readings, and verified that the exact expected sample count (40 or 200) was received.

To ensure dataset integrity, any recording that timed out or failed to meet the required sample count due to transmission errors was automatically discarded. Validated sequences were immediately saved to CSV files within a structured directory hierarchy labeled by character class.

### 3.2.3 Hypothesis Testing

To test the hypothesis, model performance was evaluated using 10-fold cross-validation. Each dataset, large slow gestures and small fast gestures, was partitioned randomly into 10 subsets (folds) that maintained class distribution. For each fold, a model was trained on all the other folds of the dataset and tested against the fold left out of training, producing 10 classification accuracies.

Each model followed the same architecture with a Global Average Pooling layer after all convolutional layers to ensure a fair comparison between the two differing numbers of samples caused by the duration of each gesture.

The architecture consisted of two convolutional blocks followed by a classification head:

- **Input Layer:** Accepts 6-channel sensor data with variable sequence length.
- **Block 1:** Two 1D Convolutional layers (32 filters, kernel size 3, ReLU activation) followed by a Max Pooling layer (pool size 2).
- **Block 2:** Two 1D Convolutional layers (64 filters, kernel size 3, ReLU activation).
- **Global Average Pooling:** This layer computes the mean of each feature map across the time dimension, condensing the variable-length input into a fixed-size vector of 64 features.
- **Classification Head:** A Dropout layer (rate 0.3) was applied to prevent overfitting, feeding into a final Dense layer with Softmax activation for the 5 character classes.

Training was performed using the Adam optimizer and Categorical Cross-Entropy loss for 50 epochs with a batch size of 16.

### 3.3 Results and Analysis

Table 3.1: Comparative classification accuracy (10 Fold CV)

| Condition  | Gesture Size | Duration | Mean Accuracy | Standard Deviation |
|------------|--------------|----------|---------------|--------------------|
| Large-Slow | ~20 cm       | ~2.0 s   | 98.80%        | 1.83%              |
| Small-Fast | ~3 cm        | ~0.4 s   | 98.80%        | 2.56%              |

To determine if the observed performance differences were statistically significant, a Corrected Paired t-test was conducted. This test is specifically designed for cross-validation results; it corrects for the violation of the independence assumption caused by the overlapping training data in k-fold cross-validation (Nadeau & Bengio, 2003).

The t-statistic (t) was calculated using the formula proposed by Nadeau and Bengio (2003):

$$t = \frac{\bar{d}}{\sqrt{\sigma^2 \left( \frac{1}{k} + \frac{n_{test}}{n_{train}} \right)}}$$

Where:

- $k$  is the number of folds (10).
- $n_{test}/n_{train}$  is the ratio of test data to training data ( $1/9 \approx 0.11$ ).
- $\bar{d}$  is the mean of the differences in accuracy between the two models across the 10 folds.
- $\sigma^2$  is the variance of these differences.

#### Calculation of Values

The classification accuracies for the Small-Fast condition and the Large-Slow condition were compared fold-by-fold.

**Mean Difference ( $\bar{d}$ ):** The average accuracy for both the Small-Fast condition (98.80%) and the Large-Slow baseline (98.80%) was identical. Consequently, the mean difference between them is exactly zero.

$$\bar{d} = 0$$

**Resultant t-value:** Since the numerator of the t-statistic equation ( $\bar{d}$ ) is zero, the calculated t-value is:

$$t = 0$$

### Critical Value Comparison

The test was performed at a significance level of 5% ( $\alpha = 0.05$ ). The degrees of freedom ( $df$ ) for a 10-fold paired test is  $k - 1 = 9$ .

According to the Student's t-distribution table, the critical value for a two-tailed test with  $df = 9$  and  $\alpha = 0.05$  is 2.262.

**Rejection Criteria:** We reject the null hypothesis if  $|t| > 2.262$ .

**Observation:** Our calculated value  $|0| < 2.262$ .

Since the calculated t-value is significantly lower than the critical value, we fail to reject the null hypothesis. This statistical result confirms with 95% confidence that there is no significant difference in model accuracy when replacing large, slow gestures with small, fast ones.

## 3.4 Outcome and Revised Project Direction

The failure to reject the null hypothesis confirmed that convenient small fast gestures were suitable for the application, and so a full dataset of small fast gestures must be collected from which the application model will be trained on.

## 4 Data Acquisition

Building on the findings from the preliminary investigation (Chapter 3.4) that small, fast gestures are not significantly more difficult to classify than larger, slower gestures at the sizes and speeds tested, this chapter details the process of creating such a dataset of small fast gesture recordings for a complete alphabet. The scripts used for this are adapted from those used in Chapter 3.2.2 to meet the requirements laid out for the dataset.

### 4.1 Dataset Requirements

#### 4.1.1 Character Set and Sample Number

In order for the system to function as a practical text input method, it must be capable of recognising a complete alphabet. To meet this requirement, the dataset covers all 26 letters of the Latin alphabet (A-Z). Capital letters were chosen for the set as uppercase gesture sets have been more widely studied in prior air-writing research, providing a basis for methodological comparison (Yanay and Shmueli (2020), Chen et al., (2016))

100 recordings were made for each of the 26 gestures, resulting in 2600 gesture samples.

#### 4.1.2 Data Labelling and Organisation

Recordings were organised into a top-level dataset directory containing one subdirectory per character class. Figure 4.1 shows the resulting folder and file naming convention.

CSV was chosen as the storage format for recordings since debugging is straightforward and parsing is trivial in Python and C++. Each CSV file contains 40 rows (one per IMU sample) and 7 columns ordered as: `acc_x`, `acc_y`, `acc_z`, `gyro_x`, `gyro_y`, `gyro_z`, `timestamp-us`. The `timestamp-us` column records microseconds elapsed since the trigger command ('R') was received, measured using the Arduino's `micros()` counter. Timestamps preserve correct sample ordering despite any serial communication irregularities and allows verification that the 100 Hz sampling rate was maintained (consecutive values should differ by roughly 10,000  $\mu$ s).

```

dataset/
|-- A/
|   |-- A_001.csv
|   |-- A_002.csv
|   |-- ...
|   '-- A_100.csv
|-- B/
|   |-- B_001.csv
|   '-- ... (100 files total)
|-- ...
'-- Z/
    |-- Z_001.csv
    '-- ... (100 files total)

```

Figure 4.1: Directory and file structure of the dataset.

### 4.1.3 Data Recording Protocol

All gestures recorded were performed by a single user using their dominant hand to control for gestural differences between different users and hands, thereby isolating character class as the only independent variable. The gestures were executed over 0.4 seconds per recording with a gesture height manually constrained to approximately 3-5 centimetres following the protocol laid out in the preliminary investigation (see Chapter 3.4).

## 4.2 Data Creation Scripts

To efficiently capture the 2600 gesture samples for the dataset, some adaptations were made to the python script used in the preliminary investigation (Section 3.2.2). The python script is primarily responsible for telling the user when to start a gesture, telling the Arduino script to start recording a gesture, time elapsed for the current gesture, and saving the gesture recording IMU data to the required directory structure.

Recording 2600 gestures is a repetitive process prone to human error and fatigue, which can negatively impact the quality of the dataset and would not match real world expected use for the application. To mitigate this, adaptations made to the script from Section 3.2.2 include batch processing, state persistence and pausing.

As opposed to recording all gestures for a character consecutively, the collection process is paused every 25 recordings forcing the user to have a rest period after which they can click to resume.

```

1 def _run_batch(self, char: str, batch_start: int) -> bool:
2     batch_end = min(batch_start + BATCH_SIZE, TOTAL_REPS)
3
4     while self.current_rep < batch_end:
5         result = self._run_one(char, self.current_rep + 1)
6
7         if result == "SUCCESS":
8             self.current_rep += 1
9             self._save_progress()
10            if self.current_rep < batch_end:
11                print("  Resting 2 s ...")
12
13            elif result == "DISCARD_PREV":
14                self._discard_last()
15                print("  Resuming batch ...")
16
17    return True

```

Listing 4.1: Batch processing logic to mitigate user fatigue during data collection.

The script maintains a progress.txt file updating the character index and repetition count after every save, allowing data collection to be spread across multiple instances of the script without losing progress or overwriting files.

```

1 def _load_progress(self):
2     if os.path.exists(self.progress_file):
3         with open(self.progress_file) as f:
4             lines = f.readlines()
5             if len(lines) >= 2:
6                 self.current_char_idx = int(lines[0].strip())
7                 self.current_rep = int(lines[1].strip())
8                 print(f"[Resumed] {CHARACTERS[self.current_char_idx]} - rep {self.
9                 current_rep + 1}/{TOTAL_REPS}")
10            else:
11                print("[New session] Starting from A, rep 1")
12
13 def _save_progress(self):
14     with open(self.progress_file, 'w') as f:
15         f.write(f"{self.current_char_idx}\n{self.current_rep}\n")

```

Listing 4.2: State persistence functions for saving and resuming data collection progress.

Due to the expected length of each batch, occasionally the user may need to stop recording the batch before it finishes, Recognising this, pause functionality was added such that if the user presses the spacebar then the latest saved CSV recording is deleted, the current recording is interrupted and not

saved and the progress file is updated, ensuring if the user is unable to complete a gesture when prompted to, the resultant recording does not enter the final dataset.

```
1 # Inside the recording cycle (_run_one)
2 self._flush_kb()
3 for tick in range(3, 0, -1):
4     if self._space_pressed():
5         print(" [SPACE] Discarding previous recording ...")
6         return "DISCARD_PREV"
7     print(f" {tick} ...", end=' ', flush=True)
8     time.sleep(1)
9
10 # Discard helper function
11 def _discard_last(self):
12     if self.current_rep <= 0:
13         return
14
15     self.current_rep -= 1
16     char = CHARACTERS[self.current_char_idx]
17     fname = f"{char}_{self.current_rep + 1:03d}.csv"
18     fpath = os.path.join(OUTPUT_DIR, char, fname)
19
20     if os.path.exists(fpath):
21         os.remove(fpath)
22     self._save_progress()
```

Listing 4.3: Countdown loop with active interruption and the corresponding rollback function.

## 4.3 Dataset Overview

Once the data collection process was complete, a dataset of 2,600 CSV files was produced. However, before the data could be fed into a convolutional neural network for training, it required preprocessing to ensure the model could learn without bias.

Because the data collection script strictly enforced a 40-sample capture window for every recording, all data sequences were naturally uniform in length. Normalisation was applied as raw accelerometer and gyroscope values have vastly different scales; the accelerometer typically outputs values between  $-30$  and  $+30 \text{ m/s}^2$ , whereas the gyroscope can record angular velocities ranging from  $-350$  to  $+350$  degrees/s (as visualised in Figure 4.2). To address this, Z-score standardisation was applied to the dataset during the model training pipeline. Each of the six sensor channels was normalised independently based on the mean and standard deviation calculated from the training set, ensuring the model treats both linear acceleration and angular velocity with equal importance.

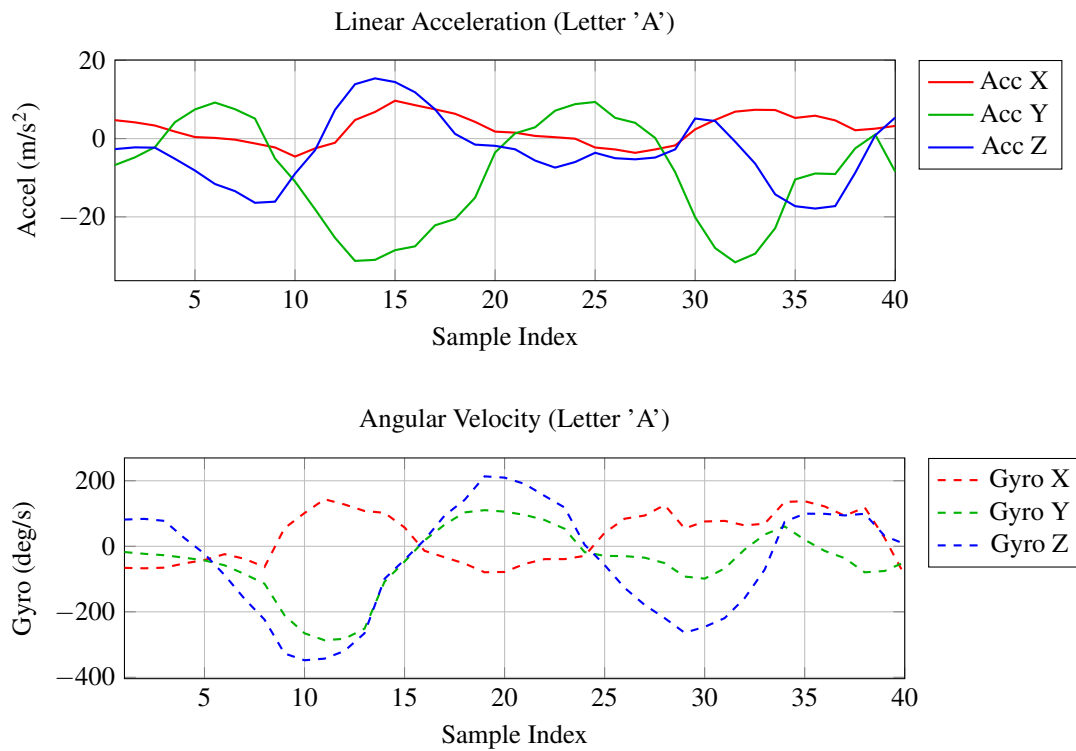


Figure 4.2: Raw 6-axis IMU data for a single capture of the letter 'A' (A\_001.csv) over the 0.4-second (40 sample) gesture window.

## 5 Model Training and Compression

This chapter describes the process of training a baseline model and application of three model optimisation techniques evaluated in this project. The baseline is an uncompressed reference point for model size and accuracy, and is used as the base model which is compressed with each technique. Pruning, quantisation and knowledge distillation are applied to the base model individually and in combination, to produce a set of models with a range of sizes, accuracies and latencies, with Keras used in model training and TensorFlow Lite used for quantisation and conversion to .tflite format.

### 5.1 Baseline Model

The baseline model is a 1D convolutional neural network consisting of three convolutional blocks followed by a classification head, defined in Listing 5.1.

```
1 model = tf.keras.Sequential([
2     tf.keras.layers.Input(shape=(40, 6)),
3
4     # Block 1
5     tf.keras.layers.Conv1D(32, kernel_size=3, activation="relu", padding="same"),
6     tf.keras.layers.Conv1D(32, kernel_size=3, activation="relu", padding="same"),
7     tf.keras.layers.MaxPooling1D(pool_size=2),
8
9     # Block 2
10    tf.keras.layers.Conv1D(64, kernel_size=3, activation="relu", padding="same"),
11    tf.keras.layers.Conv1D(64, kernel_size=3, activation="relu", padding="same"),
12    tf.keras.layers.MaxPooling1D(pool_size=2),
13
14    # Block 3
15    tf.keras.layers.Conv1D(128, kernel_size=3, activation="relu", padding="same"),
16
17    # Classification head
18    tf.keras.layers.GlobalAveragePooling1D(),
19    tf.keras.layers.Dropout(0.3),
20    tf.keras.layers.Dense(64, activation="relu"),
21    tf.keras.layers.Dense(26, activation="softmax"),
22 ])
```

---

Listing 5.1: Baseline model architecture.

Filter counts increase through the three blocks (32, 64, 128) to allow the network to learn progressively more abstract features from the raw IMU time series. Global average pooling is used rather than flattening to reduce the parameter count and provide some invariance to small timing variations in gesture execution. Dropout (0.3) is applied before the classification head to reduce overfitting on the relatively small dataset of 2,600 samples. The dataset was split into 80% training and 20% test sets using stratified random sampling with a fixed random seed of 42. Z-score normalisation was applied per feature channel, with the mean and standard deviation computed from the training set only and saved for use during on-device inference. The model was trained using the Adam optimiser with categorical cross-entropy loss for up to 50 epochs with a batch size of 16. Early stopping with a patience of 10 epochs monitored validation accuracy and restored the best weights. The trained baseline achieves 98.27% accuracy on the held-out test set, with 56,922 parameters and a TensorFlow Lite model size of 230.3 KB. These figures serve as the reference point against which all compressed variants are compared.

## 5.2 Model Compression Methodology

### 5.2.1 Structured Pruning

Structured pruning was applied to models by removing entire convolutional filters and dense layer neurons ranked by L1-norm importance, following the approach of Li et al. (2017). For each convolutional layer, the importance of each filter is computed as the sum of absolute values of its weights. Filters with the lowest importance scores are removed, and the corresponding input channels of the subsequent layer are adjusted to maintain architectural consistency.

```
1 def compute_filter_importance(kernel_weights):  
2     return np.sum(np.abs(kernel_weights), axis=(0, 1))  
3  
4 def compute_neuron_importance(dense_weights):  
5     return np.sum(np.abs(dense_weights), axis=0)
```

Listing 5.2: L1-norm importance scoring for convolutional filters and dense neurons.

For each sparsity level, the number of filters to retain in each layer is calculated as the original filter count multiplied by (1 - sparsity), with a minimum of one filter per layer. The pruning is applied uniformly across all layers at the same sparsity ratio. The final classification layer is not pruned, as reducing its output dimensionality would change the number of classes the model can predict.

```

1 def prune_conv_layer(kernel, bias, num_to_keep, prev_keep_indices=None):
2     if prev_keep_indices is not None:
3         kernel = kernel[:, prev_keep_indices, :]
4
5     importance = compute_filter_importance(kernel)
6     keep_indices = np.argsort(importance)[-num_to_keep:]
7     keep_indices = np.sort(keep_indices)
8
9     pruned_kernel = kernel[:, :, keep_indices]
10    pruned_bias = bias[keep_indices]
11
12    return pruned_kernel, pruned_bias, keep_indices

```

Listing 5.3: Filter selection and pruning for a single convolutional layer.

After pruning, the reduced architecture is rebuilt as a new Keras model and the retained weights are loaded into it. The pruned model is then fine-tuned on the training data for up to 20 epochs with a reduced learning rate of  $1 \times 10^{-4}$  to recover accuracy lost from filter removal, using early stopping with a patience of 8 epochs. Pruning was evaluated at sparsity levels of 30%, 40%, 50%, 60%, 70%, 80%, 90%, and 95% to characterise the relationship between compression ratio and accuracy degradation.

### 5.2.2 Quantisation

Post-training int8 quantisation was applied to models using the TensorFlow Lite converter. This reduces all weights and activations from 32-bit floating point to 8-bit integer representation. Activation tensors require calibration data to determine the appropriate scaling ranges for each layer (see Section 2.4). To calibrate these ranges, a representative dataset of 200 randomly sampled training examples was run through the model during conversion, allowing the converter to record activation ranges at each layer and compute the scaling factors needed for int8 inference.

```

1 def quantise_int8(model, output_path, representative_dataset_fn):
2     converter = tf.lite.TFLiteConverter.from_keras_model(model)
3     converter.optimizations = [tf.lite.Optimize.DEFAULT]
4     converter.representative_dataset = representative_dataset_fn
5     converter.target_spec.supported_ops = [
6         tf.lite.OpsSet.TFLITE_BUILTINS_INT8
7     ]
8     converter.inference_input_type = tf.int8
9     converter.inference_output_type = tf.int8
10    tflite_model = converter.convert()
11
12    with open(output_path, "wb") as f:

```

```
13 f.write(tflite_model)
```

Listing 5.4: Int8 quantisation using the TensorFlow Lite converter with representative dataset calibration.

Setting `inference_input_type` and `inference_output_type` to `int8` ensures the model accepts and produces integer values at its boundaries. The `TFLITE_BUILTINS_INT8` operator set restricts the converted model to operations that are fully supported in integer-only execution on the Cortex-M4F.

### 5.2.3 Knowledge Distillation

The trained baseline model was used as the teacher for knowledge distillation, as described in Section 2.4. Three student architectures of increasing capacity were designed to evaluate distillation effectiveness across different model sizes.

```
1 STUDENT_CONFIGS = {
2     "small": [ # 1,802 params
3         ("Conv1D", {"filters": 16, "kernel_size": 3, "activation": "relu", "padding": "same"}),
4         ("Conv1D", {"filters": 16, "kernel_size": 3, "activation": "relu", "padding": "same"}),
5         ("MaxPooling1D", {"pool_size": 2}),
6         ("GlobalAveragePooling1D", {}),
7         ("Dropout", {"rate": 0.3}),
8         ("Dense", {"units": 16, "activation": "relu"}),
9     ],
10    "medium": [ # 7,674 params
11        ("Conv1D", {"filters": 16, ...}), ("Conv1D", {"filters": 16, ...}),
12        ("MaxPooling1D", {"pool_size": 2}),
13        ("Conv1D", {"filters": 32, ...}), ("Conv1D", {"filters": 32, ...}),
14        ("MaxPooling1D", {"pool_size": 2}),
15        ("GlobalAveragePooling1D", {}),
16        ("Dropout", {"rate": 0.3}),
17        ("Dense", {"units": 32, "activation": "relu"}),
18    ],
19    "big": [ # 14,906 params
20        ("Conv1D", {"filters": 16, ...}), ("Conv1D", {"filters": 16, ...}),
21        ("MaxPooling1D", {"pool_size": 2}),
22        ("Conv1D", {"filters": 32, ...}), ("Conv1D", {"filters": 32, ...}),
23        ("MaxPooling1D", {"pool_size": 2}),
24        ("Conv1D", {"filters": 64, ...}),
25        ("GlobalAveragePooling1D", {}),
26        ("Dropout", {"rate": 0.3}),
27        ("Dense", {"units": 32, "activation": "relu"}),
```

```

28     ],
29 }

```

Listing 5.5: Student model architectures for knowledge distillation.

The student architectures follow the same general pattern as the baseline with convolutional blocks followed by global average pooling and a classification head but with reduced filter counts and, for the small student, fewer convolutional blocks. The aim of this was to make performance differences be caused primarily by model capacity rather than architectural changes.

Distillation was implemented using a training loop adapted from Borup’s (2020) Keras knowledge distillation example. During each training step, the teacher’s output and the student’s output are both softened by dividing by the temperature parameter before applying softmax. The student’s loss combines two terms: a standard cross-entropy loss against the true labels, and a divergence term measuring the distance between the teacher’s and student’s softened output (see Section 2.4).

```

1  def train_step(self, data):
2      x, y = data
3      teacher_pred = self.teacher(x, training=False)
4
5      with tf.GradientTape() as tape:
6          student_pred = self.student(x, training=True)
7
8          student_loss = self.student_loss_fn(y, student_pred)
9          distillation_loss = self.distillation_loss_fn(
10              tf.nn.softmax(teacher_pred / self.temperature, axis=1),
11              tf.nn.softmax(student_pred / self.temperature, axis=1),
12              ) * (self.temperature ** 2)
13
14          loss = self.alpha * student_loss + (1 - self.alpha) *
15              distillation_loss
16
17          gradients = tape.gradient(loss, self.student.trainable_variables)
18          self.optimizer.apply_gradients(zip(gradients, self.student.
19              trainable_variables))

```

Listing 5.6: Core distillation training step showing dual-loss computation adapted from Borup, 2020.

## 6 Model Testing and Evaluation

In this chapter the accuracy and size of all resultant models, from the application of optimisation techniques to the baseline model as described in Section 5.2, is measured and contextualised, models that met a specific accuracy threshold were deployed on to the Arduino Nano 33 BLE where on-device accuracy, inference latency, and tensor arena memory usage were measured.

### 6.1 Performance Results

#### 6.1.1 Accuracy vs Size Across Compression Techniques

The results in this section were obtained by evaluating each compressed model's .tflite file using the TensorFlow Lite interpreter on the host machine against the held-out test set (520 samples). On-device results from the Arduino Nano 33 BLE Sense are presented separately in Section 6.3 for selected models.

Table 6.1 and Figure 6.1 show the accuracy and model size of the baseline model after structured pruning at sparsity levels from 30% to 95%. Accuracy degrades gradually up to 60% sparsity, where the model retains 90.19% accuracy while reducing in size from 230.3KB to 44.4KB, an 81% reduction in model size. Beyond this point, model accuracy drops sharply; at 70% sparsity only 74.23% accuracy is retained, and further than this the model effectively fails. This suggests a threshold between 70–80% sparsity at which the remaining filters in each convolutional layer can no longer learn sufficient distinct feature representations to discriminate between 26 character classes.

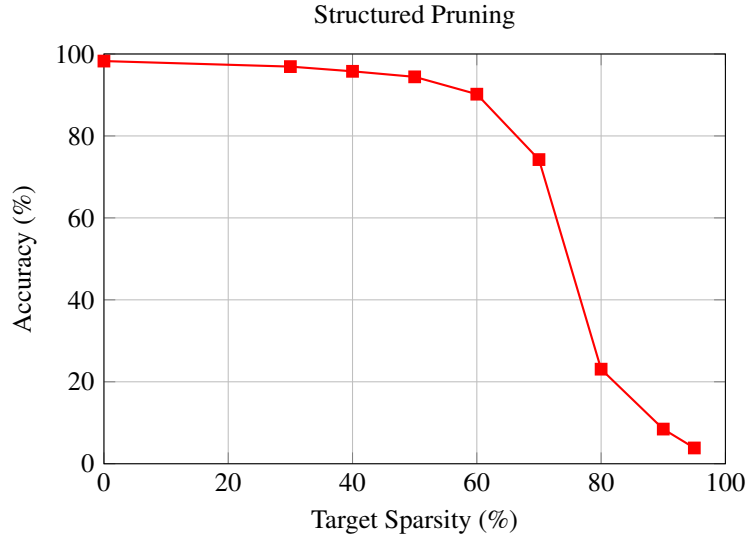


Figure 6.1: Degradation of accuracy under structured pruning.

Table 6.1: Structured pruning results.

| Sparsity | Params | Size (KB) | Accuracy (%) |
|----------|--------|-----------|--------------|
| Baseline | 56,922 | 230.3     | 98.27        |
| 30%      | 27,659 | 115.7     | 96.92        |
| 40%      | 20,717 | 88.7      | 95.77        |
| 50%      | 14,906 | 66.1      | 94.42        |
| 60%      | 9,349  | 44.4      | 90.19        |
| 70%      | 5,522  | 29.4      | 74.23        |
| 80%      | 2,475  | 17.5      | 23.08        |
| 90%      | 749    | 10.8      | 8.46         |
| 95%      | 250    | 8.8       | 3.85         |

Figure 6.2 shows the accuracy of three student architectures trained both standalone, directly on the training dataset, and with knowledge distillation using the baseline model as a teacher. Only the big distilled student model outperformed its standalone counterpart; as seen in Table 6.2 the big student model gained 2.31 percentage points of accuracy from distillation with a 71% size reduction over the baseline. The medium student model performed slightly better when trained directly on the training data, but the small model performed significantly better standalone, with distilled only retaining 78.27% accuracy compared to 90% when trained directly on the training dataset. This may be due

to smaller students lacking the sufficient capacity to represent the teacher’s output distribution; the richer information in the soft targets can only be useful if the student model has the capacity to learn these features (Cho and Hariharan, 2019).

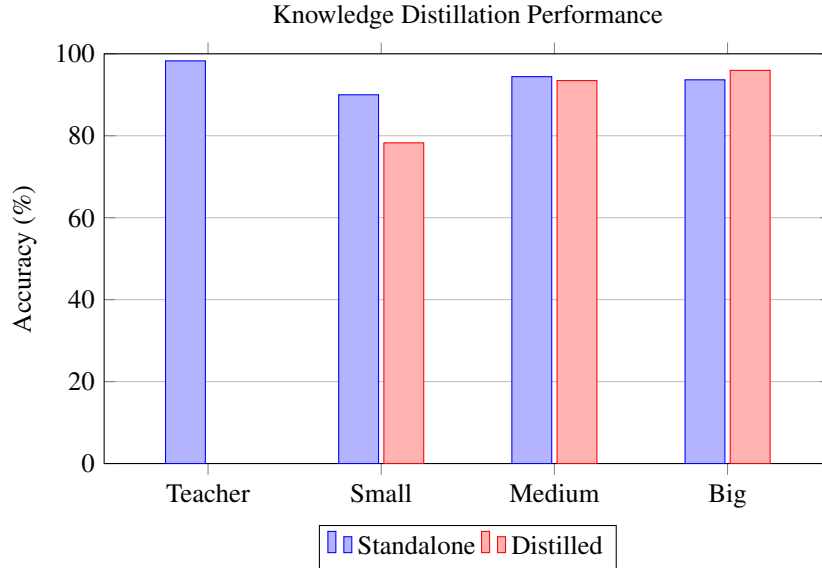


Figure 6.2: Comparison of standalone student accuracy versus accuracy after distillation.

Table 6.2: Knowledge distillation results across student sizes.

| Student | Params | Standalone (%) | Distilled (%) | KD Gain (pp) | Size (KB) |
|---------|--------|----------------|---------------|--------------|-----------|
| Teacher | 56,922 | 98.27          | —             | —            | 230.3     |
| Small   | 1,802  | 90.00          | 78.27         | −11.73       | 12.1      |
| Medium  | 7,674  | 94.42          | 93.46         | −0.96        | 37.7      |
| Big     | 14,906 | 93.65          | 95.96         | +2.31        | 66.9      |

To explore whether combining compression methods could yield superior results, structured pruning was applied to the successfully distilled big student model. As detailed in Table 6.3 and illustrated in Figure 6.3, the hybrid approach (KD + Pruning) demonstrated a much steeper degradation curve than structured pruning alone. Removing 40% of the filters from the distilled student reduced its size by 36.8KB, but accuracy fell to 87.69%, significantly lower than the 95.96% accuracy of the big distilled model. Beyond 50% sparsity, accuracy loss is extreme, putting the threshold at which the remaining filters are not sufficient to learn the feature representations to effectively discriminate between classes between 50-60%, compared to 60-70% for the baseline model pruning.

This indicates that while distillation is highly effective at teaching a compact architecture to retain high accuracy, the resulting model becomes "dense" with learned representations; its filters contain critical information. Therefore, applying structured pruning to an already distilled model quickly removes these vital feature representations, leading to rapid accuracy degradation.

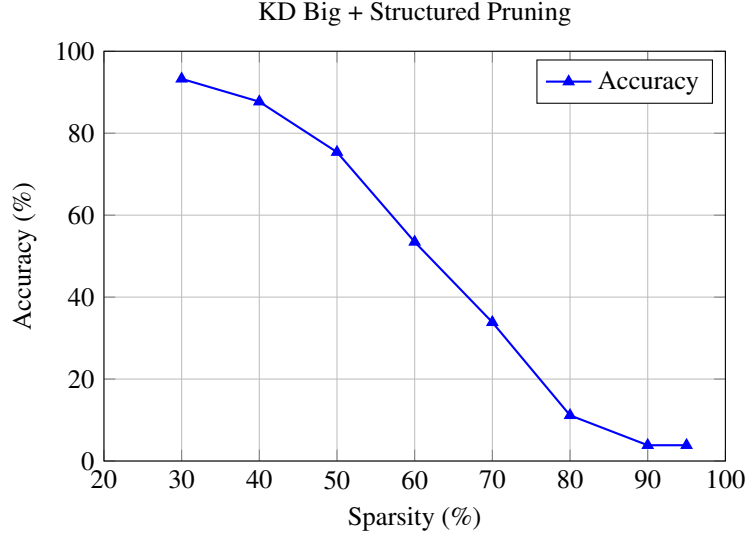


Figure 6.3: Accuracy of the KD *big* student after further structured pruning.

Table 6.3: Knowledge distillation combined with structured pruning, applied to the *big* student.

| Sparsity | Params | Size (KB) | Accuracy (%) |
|----------|--------|-----------|--------------|
| 30%      | 7,341  | 37.2      | 93.27        |
| 40%      | 5,522  | 30.1      | 87.69        |
| 50%      | 4,074  | 24.4      | 75.38        |
| 60%      | 2,475  | 18.2      | 53.46        |
| 70%      | 1,469  | 14.3      | 33.85        |
| 80%      | 749    | 11.5      | 11.15        |
| 90%      | 250    | 9.6       | 3.85         |
| 95%      | 99     | 8.9       | 3.85         |

### 6.1.2 Model Selection

Of the 19 optimised models produced, a subset were selected for on-device testing for which accuracy, latency and tensor arena usage were tested on the test set. The threshold chosen to select

models was an accuracy greater than 90% as shown in Figure 6.4, this was chosen as for the air writing recognition use case, lower accuracy can in cases be made up for by post inference correction (see Section 2.2) and the variety of modes includes all optimisation techniques. Also included for on-device testing was the baseline model, the big distilled model, and the distilled model pruned to 30% sparsity level, all quantised from float32 to int8 after training following the methodology set out in Section 5.2.2.

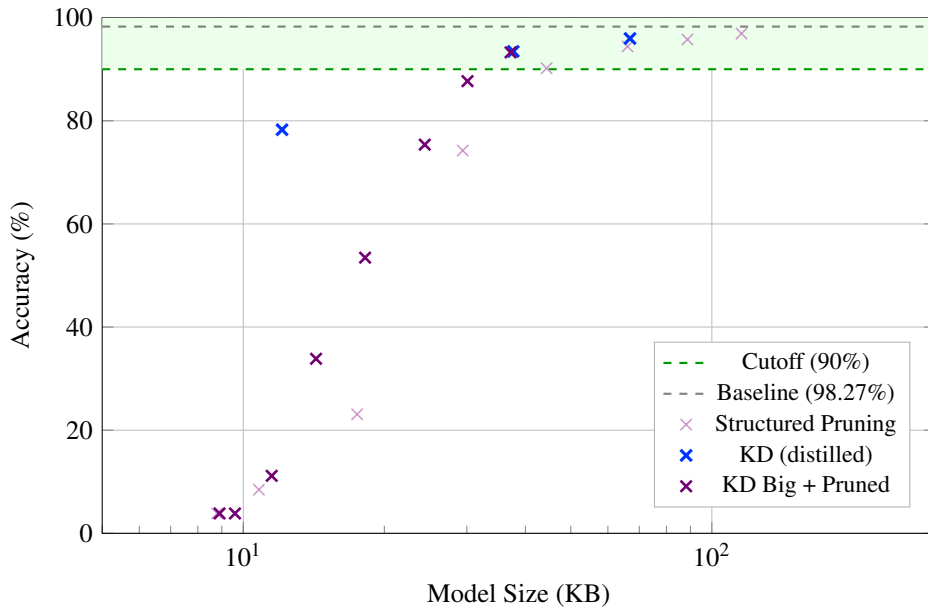


Figure 6.4: Accuracy versus model size (KB) on a logarithmic scale

### 6.1.3 Testing methodology

Each compressed model was evaluated on the Arduino Nano 33 BLE Sense to measure on-device accuracy, inference latency, and tensor arena memory usage. This section describes the deployment and evaluation process.

To deploy a TensorFlow Lite model to the Arduino, the .tflite file is converted to a C byte array stored in flash memory. The Arduino sketch loads this array at startup, initialises the TFLite Micro interpreter, allocates the tensor arena, and reports the arena usage in bytes. The sketch auto-detects whether the loaded model expects float32 or int8 input tensors and reports the input type, scale, and zero point to the host over serial, allowing the evaluation script to format data appropriately.

Evaluation was conducted by sending the held-out test set (20% of the dataset, 520 samples) from a Python script to the Arduino over a serial connection. For each sample, the script transmits the

40 timesteps of normalised 6-axis IMU data one row at a time. For int8 models, the Python script pre-quantises each value using the model’s reported scale and zero point before transmission. Once all 40 timesteps are received, the sketch copies the data into the input tensor, invokes the interpreter, and returns the predicted class index and the inference duration in microseconds measured using the Arduino’s `micros()` counter.

The test set used for on-device evaluation is identical to the test set used during off-device training evaluation with the same 80/20 stratified split with the same random seed. This ensures that on-device accuracy figures are directly comparable to the accuracies reported during model training and compression. No training data was used during on-device evaluation.

#### **6.1.4 On-device Performance of Selected Models**

The size of the .tflite file dictates how much Flash memory is consumed by the model’s static weights, biases, and schema. However, this metric does not account for the memory required during inference. In TensorFlow Lite for Microcontrollers (TFLM), runtime memory is managed via a statically allocated block of SRAM known as the Tensor Arena. The Tensor Arena provides the intermediate buffers required to hold input tensors, output tensors, and the temporary activation values generated as data passes through the network’s layers (David et al., 2021).

Table 6.4 details the on-device accuracy and Tensor Arena requirements for the selected models. Figure 6.5 visualises this trade-off, demonstrating how different compression techniques impact the runtime RAM footprint.

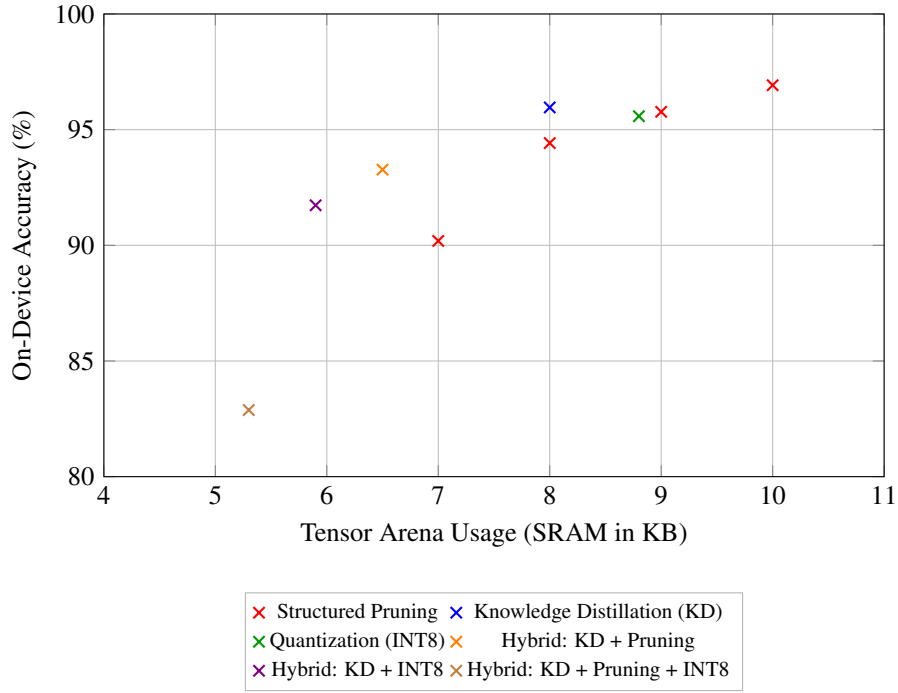


Figure 6.5: On-device Accuracy versus Tensor Arena RAM usage split by compression technique combinations.

Analysing the Arena usage reveals how different techniques interact with SRAM. Structured pruning reduces the number of filters per layer (see Section 2.4), directly shrinking the output feature maps and the memory required to buffer them and so the pruned models scale linearly from 10.0KB at 30% sparsity to 7.0KB at 60%. Int8 quantisation achieves a similar reduction without altering the architecture, as activations are buffered as 8-bit integers rather than 32-bit floats; the baseline model's Arena dropped to 8.8KB when quantised. Compounding techniques yielded the lowest Arena footprints with the KD big + int8 model using just 5.9KB however, the impact on accuracy for further compounding is severe (the relationship between compounding and accuracy is discussed further in Section 6.2).

Table 6.4: On-device accuracy and tensor arena usage on the Arduino Nano 33 BLE.

| Model                      | Accuracy (%) | Arena (KB) |
|----------------------------|--------------|------------|
| KD big                     | 95.96        | 8.0        |
| KD big + pruned 30%        | 93.27        | 6.5        |
| Structured pruned 30%      | 96.92        | 10.0       |
| Structured pruned 40%      | 95.77        | 9.0        |
| Structured pruned 50%      | 94.42        | 8.0        |
| Structured pruned 60%      | 90.19        | 7.0        |
| KD big + pruned 30% + int8 | 82.88        | 5.3        |
| KD big + int8              | 91.73        | 5.9        |
| Baseline + int8            | 95.58        | 8.8        |

Beyond application responsiveness, inference latency is a factor in the viability of battery-powered TinyML applications. As noted by Warden and Situnayake (2019), the time taken to perform a single inference is a fundamental component in estimating the total energy usage of a system. Consequently, lower latency provides a more responsive user experience while simultaneously enabling a more energy-efficient application.

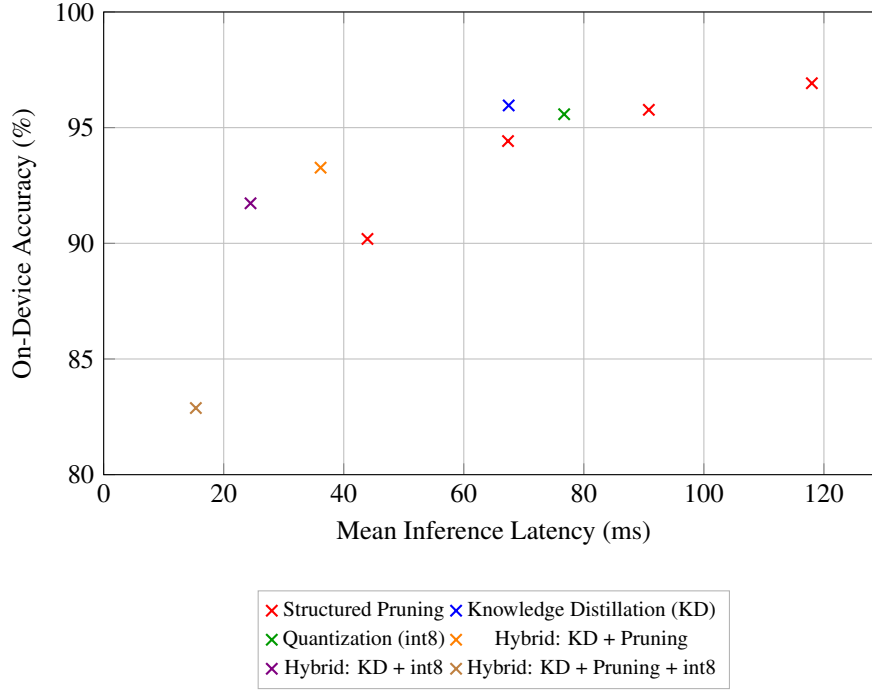


Figure 6.6: On-device Accuracy versus Inference Latency (ms) split by compression technique combinations.

The results in Table 6.5 and Figure 6.6 demonstrate that compression techniques significantly improve inference latency. As filters were removed from the convolutional layers and so the total number of operations decreased, structured pruning led to a reduction in latency from 117.99 ms at 30% sparsity to 43.95 ms at 60% sparsity.

The lowest inference latency, as with Tensor Arena Usage, was achieved by the hybrid distilled, pruned and quantised model, with 15.36 ms average latency at the cost of severe accuracy loss (as seen in Figure 6.6). In contrast, the distilled student model when only quantised or only pruned to 30% sparsity level retained an accuracy over 90% whilst significantly reducing model latency over the distilled model alone. Quantisation of weights to int8 results in much simpler calculations for the MCU to perform, allowing for a large drop in latency from 67.48 ms to 24.47 ms for the distilled model.

Table 6.5: On-device inference latency on the Arduino Nano 33 BLE.

| Model                      | Mean (ms) | Range (ms) |
|----------------------------|-----------|------------|
| KD big                     | 67.48     | 0.12       |
| KD big + pruned 30%        | 36.14     | 0.08       |
| Structured pruned 30%      | 117.99    | 0.18       |
| Structured pruned 40%      | 90.83     | 0.18       |
| Structured pruned 50%      | 67.37     | 0.15       |
| Structured pruned 60%      | 43.95     | 0.12       |
| KD big + pruned 30% + int8 | 15.36     | 0.23       |
| KD big + int8              | 24.47     | 0.29       |
| Baseline + int8            | 76.72     | 0.30       |

## 6.2 Evaluation of Model Trade-offs

The selection of an "optimal" model for air-writing recognition requires a hierarchy of prioritisation between system accuracy, inference latency (or energy consumption) and memory footprint (max RAM usage and flash memory usage), as demonstrated by the wide range of models produced through the optimisation processes which all have distinct benefits and costs.

If the primary objective is to minimise latency or energy consumption, an excellent choice would be a combination of knowledge distillation and int8 quantisation. The model achieved 24.47 ms latency at the cost of a slight reduction in accuracy and a comparatively very small amount of maximum SRAM usage. If the usage of the application is real time communication or a battery powered general text input device with Bluetooth, this model provides a good balance of accuracy and memory footprint. However, under circumstances where text inputted can't be easily corrected after misclassifying a gesture, or post inference correction couldn't be applied to correct word spelling, 91.73% accuracy may be insufficient and a trade-off must be made on inference latency, choosing between device longevity and responsiveness and user frustration/inconvenience. The big distilled model at float32 may be more effective despite a latency increase to 67.48 ms in such a case.

As with inference latency, knowledge distillation quantised to int8 proved particularly well suited to situations with low RAM usage requirements; 5.9KB of the Tensor Arena were used at maximum usage. We can see why this combination is so well suited to these metrics in how they perform alone and the way in which they function. Structured Pruning of the baseline model at 30% sparsity represents the baseline model with significant optimisation, which we can see from the size drop from 230.3KB to 115.7KB. If we take the maximum tensor arena usage and the average inference latencies of the pruned baseline model at 30% sparsity as an underestimate for the standalone baseline

models hypothetical performance, we can see how knowledge distillation alone contributes significantly to the optimisation of the KD int8 model. 8KB of max arena tensor usage vs 10KB for the baseline underestimate, and 67.48 ms vs 117.99 ms for the baseline latency underestimate. This can be explained by the gap in parameters between the two models (see Table 6.2), with distillation allowing for the accuracy loss to remain minimal with a much smaller architecture.

Following this, quantisation removes redundancy from float32 weight precision but the benefits of int8 quantisation in terms of latency is primarily due to the conversion of weights to a much simpler data type. The MCU can execute integer arithmetic with significantly fewer clock cycles than floating point operations. RAM usage is optimised simply due to the reduction in size of weights, from 32 bit to an 8 bit data type, the size of the weights and activations being used is reduced by 75%, resulting in an Tensor Arena footprint reduction from 8.0KB to 5.9KB.

A model which performs very closely to the high benchmark of distillation quantised to int8 is the big distilled model structurally pruned to 30% sparsity. At the cost of 0.6KB maximum tensor arena usage and 11.67 ms of average latency the model has a 1.54% higher accuracy on the test set. In the vast majority of use cases the latency loss and extra RAM usage are negligible and so the model is very well suited for a system where higher accuracy is very valuable.

Of the optimised models, the one with the highest accuracy was the baseline model pruned to 30% sparsity level with 96.92% performance on the test set. One notable set of results is the difference between the distilled model and the baseline model pruned to 50% sparsity. Both have a maximum tensor arena usage of exactly 8.0KB, a latency of 67 ms rounded to two significant figures, and accuracies of 95.96% and 94.42% respectively. Furthermore, the distilled model has a flash footprint of 66.9KB vs 66.1KB for the pruned baseline, acting as a case in which one compression technique directly outperforms another in a single metric with all other performance results incredibly similar. The only cause for the difference in accuracy is the way in which the model is optimised but not through its effect on model size.

In summary, no single compression technique dominates across all metrics; the choice of model depends on the deployment constraints and priorities of the target application. For air-writing text input on the Arduino Nano 33 BLE Sense, where gestures are performed sequentially with natural pauses between characters, inference latencies below 120ms are imperceptible to the user, making accuracy the primary concern. Under this criterion, the big distilled student model offers the strongest balance: 95.96% accuracy in a 66.9KB flash footprint with 8.0KB tensor arena usage and 67ms latency, achieving a 71% reduction in model size over the baseline with only a 2.31 percentage point accuracy loss. Where RAM availability is constrained by concurrent processes such as a potential Bluetooth stack, the distilled model quantised to int8 provides a viable alternative at 91.73% accuracy with a tensor arena footprint of just 5.9KB. The results demonstrate that for this task, knowledge distil-

lation into a well-sized student architecture consistently produced the most favourable deployment profiles, and that combining distillation with either pruning or quantisation extends the achievable range of size-accuracy-latency trade-offs to suit a variety of resource budgets.

## 7 Conclusion

### 7.1 Summary and Key Findings

With this project we have demonstrated the potential for an application deployed on a microcontroller that can classify a full alphabet of convenient, small fast gestures. In all cases, structured pruning provided significant optimisation benefits up until a size threshold after which accuracy degradation was catastrophic for real world deployment. Quantisation proved valuable for models that weren't already hyperoptimised, providing strong benefits to inference latency and both measures of memory footprint. Knowledge distillation demonstrated the capability for a student model to learn data features better from the softened output distribution of a teacher model than directly from only the raw data truth labels, however this only showed benefit when model size was not small enough to lack the capacity to learn in this way. In cases where the model was too small, distillation had a negative effect on learning, resulting in significantly worse accuracy outcomes. The big distilled student model, and variants of, were found to offer the best range of tradeoffs between accuracy, memory and latency and so for a use case in which classification mistakes can be mitigated with automated spelling checks post inference (see Section 2.2), knowledge distillation is the recommended technique based on the results of this project.

### 7.2 Reflection on Project Aims

Project aims taken directly from Section 1.1.

(1) investigate whether small, fast gestures can achieve classification accuracy comparable to the larger, slower gestures used in prior work, validating their use for practical text input

This objective was largely achieved, through hypothesis testing it has been demonstrated that small, fast gestures can be classified with a statistically insignificant difference in accuracy to large slow gestures on a similar model architecture. However, the datasets are limited in a few ways: the user recording gestures was stationary, outside of the gestural motion, the gesture duration was kept constant and the user was aware of when the gesture would finish recording. As well as this, only one hardware platform, model architecture and user were used in the process. This prevents the result of the hypothesis test from being fully generalisable, but through the results of the optimised model deployment we can show that a TinyML application that can classify user-dependent microgestures is achievable, even on extremely resource constrained edge devices.

(2) create a dataset of 26 air-written characters using small, fast gestures captured from 6-axis IMU

data

This objective was completed successfully, previously building very similar scripts for data collection purposes during hypothesis testing made the process significantly easier as the main development focus during data acquisition was scaling the host computer script to allow the user to take breaks for such a large dataset to record. The core challenge of the process was the length of time required to record the quantity of recordings I had found suitable for the project.

(3) train a baseline 1D CNN model and apply post-training quantisation, structured pruning, and knowledge distillation individually and in combination

The process of doing this proved difficult due to the range of understanding required to develop scripts to implement the optimisation scripts, however the objective was successfully achieved and a wide range of effective models was produced providing a sufficient basis for evaluation of each technique.

(4) evaluate the compressed models on-device across accuracy, model size, RAM usage, and inference latency to determine which compression strategies offer the most effective tradeoffs for microcontroller deployment for the use case.

This objective was partially achieved, however a large limitation proved to be the available hardware and software choice limiting the range of applicable optimisation techniques. Unstructured pruning and quantisation to different levels such as float16 and int4 would have provided a stronger and more generalisable evaluation of which optimisation techniques are most effective for the usecase but don't function within the tensorflow ecosystem or can't be deployed onto the Arduino Nano 33 BLE. Despite this, a large variety of models were benchmarked across all performance metrics which allowed for an evaluation of each model optimisation technique available and how they relate to the tradeoffs central to TinyML deployment.

## 7.3 Recommendations for Future Work

Based on the limitations of this study, an evaluation of different model optimisation techniques across hardware platforms for IMU based gesture classification on an MCU would be a valuable result towards generalising the finding that knowledge distillation was most effective for the usecase, allowing for further optimisation techniques to be evaluated such as quantisation aware training, low rank factorisation and unstructured pruning.

Another area in which research focus would expand the direction this project takes would be evaluating the feasibility of microgestures for continuous input sentence level recognition with inference completed on an edge device, producing a fully functional text-input tool. For single character recognition, the application of on device learning to fine tune the model to classify the air-writing gestures

of an individual user after training would allow the system to adapt to individual handwriting styles without requiring retraining on a host machine, improving user-independent classification accuracy while maintaining the privacy and connectivity advantages of on-device inference.

## References

- [1] Aggarwal, P., Syed, Z., Niu, X. and El-Sheimy, N. (2008) ‘A standard testing and calibration procedure for low cost MEMS inertial sensors and units’, *Journal of Navigation*, 61(2), pp. 323–336.
- [2] Al Abir, F., Al Siam, M., Sayeed, A., Hasan, M.A.M. and Shin, J. (2021) ‘Deep learning based air-writing recognition with the choice of proper interpolation technique’, *Sensors*, 21(24), 8407.
- [3] Amma, C., Georgi, M. and Schultz, T. (2014) ‘Airwriting: a wearable handwriting recognition system’, *Personal and Ubiquitous Computing*, 18(1), pp. 191–203.
- [4] Arduino (n.d.) *Arduino Nano 33 BLE Sense Datasheet (ABX00031)*. Available at: <https://docs.arduino.cc/resources/datasheets/ABX00031-datasheet.pdf> (Accessed: February 2026).
- [5] Borup, K. (2020) *Knowledge Distillation*. Available at: [https://keras.io/examples/vision/knowledge\\_distillation/](https://keras.io/examples/vision/knowledge_distillation/) (Accessed: February 2026).
- [6] Chen, M., AlRegib, G. and Juang, B.-H. (2016) ‘Air-writing recognition—Part I: Modeling and recognition of characters, words, and connecting motions’, *IEEE Transactions on Human-Machine Systems*, 46(3), pp. 403–413.
- [7] Cheok, M.J., Omar, Z. and Jaward, M.H. (2019) ‘A review of hand gesture and sign language recognition techniques’, *International Journal of Machine Learning and Cybernetics*, 10(1), pp. 131–153.
- [8] Cho, J.H. and Hariharan, B. (2019) ‘On the efficacy of knowledge distillation’, *arXiv preprint arXiv:1910.01348*.
- [9] David, R., Duke, J., Jain, A., Reddi, V.J., Jeffries, N., Li, J., Kreeger, N., Nappier, I., Natraj, M., Regev, S., Rhodes, R., Wang, T. and Warden, P. (2021) ‘TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems’, *Proceedings of Machine Learning and Systems (MLSys)*, 3, pp. 800–811.
- [10] Hinton, G., Vinyals, O. and Dean, J. (2015) ‘Distilling the knowledge in a neural network’, *arXiv preprint arXiv:1503.02531*.
- [11] Jacob, B., Kligys, S., Chen, B., Zhu, M., Tang, M., Howard, A., Adam, H. and Kalenichenko, D. (2018) ‘Quantization and training of neural networks for efficient integer-arithmetic-only

- inference’, *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2704–2713.
- [12] Lamaakal, I., Yahyati, C., Ouahbi, I., Makkaoui, K.E. and Maleh, Y. (2025) ‘A Survey of Model Compression Techniques for TinyML Applications’, *2025 International Conference on Circuit, Systems and Communication (ICCSC)*, pp. 1–6.
  - [13] Li, H., Kadav, A., Durdanovic, I., Samet, H. and Graf, H.P. (2017) ‘Pruning Filters for Efficient ConvNets’, *Proceedings of the 5th International Conference on Learning Representations (ICLR)*.
  - [14] Nadeau, C. and Bengio, Y. (2003) ‘Inference for the generalization error’, *Machine Learning*, 52(3), pp. 239–281.
  - [15] Saha, S.S., Sandha, S.S. and Srivastava, M. (2022) ‘Machine Learning for Microcontroller-Class Hardware: A Review’, *IEEE Sensors Journal*, 22(22), pp. 21362–21390.
  - [16] Viswanatha, V., Ramachandra, A.C., Prasanna, R.R., Kakarla, P.C., Simha, P.J.V. and Mohan, N. (2022) ‘Implementation of Tiny Machine Learning Models on Arduino 33 BLE for Gesture and Speech Recognition’, *arXiv preprint arXiv:2207.12866*.
  - [17] Warden, P. and Situnayake, D. (2019) *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. Sebastopol: O’Reilly Media.
  - [18] Yanay, T. and Shmueli, E. (2020) ‘Air-writing recognition using smart-bands’, *Pervasive and Mobile Computing*, 66, 101183.